



Team Name: IslandSquad

Team Members: Laura Conover, Thomas Jaymon, Denae Liong

Course: ITAI 1378 Computer Vision and AI

Tier: Tier 1

Project Title: Animals-10 Species Classifier Proof of Concept

The Problem (30-Second Pitch)

- Problem: Identifying animal species in raw field images is challenging due to varying backgrounds, lighting conditions, and non-English directory naming conventions in raw public datasets.
- Real-World Relevance: Automated wildlife monitoring, eco-tourism applications, and automated sorting of biological field surveys.
- Goal: Deliver an automated, lightweight image-classification pipeline capable of handling 10 animal species with fast inference on real-world inputs.





What You Built (System Overview)

- One-Sentence Solution: A lightweight, end-to-end computer vision pipeline using

MobileNetV2 transfer learning to automatically clean, classify, and evaluate 10 animal species

from real-world photograph inputs.

- System Flow Diagram:
- Animal Image
- ↓
- Image Preprocessing
- ↓
- MobileNetV2 Model
- ↓
- Prediction
- ↓
- Animal Species + Confidence Score

How It Works (Technical Architecture)

- Technical Architecture & Strategy
- Base Architecture: MobileNetV2 initialized with ImageNet pre-trained weights.
- Data Augmentation: In-pipeline Keras layers (RandomFlip, RandomRotation(0.15), RandomZoom(0.15), RandomContrast(0.1)).
- Two-Phase Transfer Learning: * Phase 1 (Feature Extraction): Base layers frozen; classifier head trained for 10 epochs (LR= 10^{-3}).
 - Phase 2 (Fine-Tuning): Unfroze top 30 layers; trained for 10 epochs (LR= 10^{-5}) using ReduceLROnPlateau and EarlyStopping.

System Demo

- Proof of Concept Demonstration
- Video
Link: https://drive.google.com/file/d/1OXKiL_Txa8mB2kx9oMELbQ5Cg7GQVhQk/view?usp=sharing
- Key Highlights of the System:
- Preprocesses real inputs (224×224 resolution) and runs inference in under 25ms.
- Displays predicted species along with confidence score percentages.
- Demonstrates transparent edge-case failure modes (e.g., cow vs. horse visual overlap).

The Data You Used

- Dataset Management & Preprocessing
- Source: Animals-10 dataset from Kaggle ([alessiocrrado99/animals10](#)).
- Label Cleaning: Automated translation map from raw Italian directory names (cane → dog, ragno → spider, pecora → sheep, etc.).
- Data Selection: Sampled a balanced subset of 500 images per class (5,000 total clean images).
- Splits: * 80% Training: 4,000 images
 - 10% Validation: 500 images
 - 10% Test (Isolated): 500 images

Results vs. Your Targets

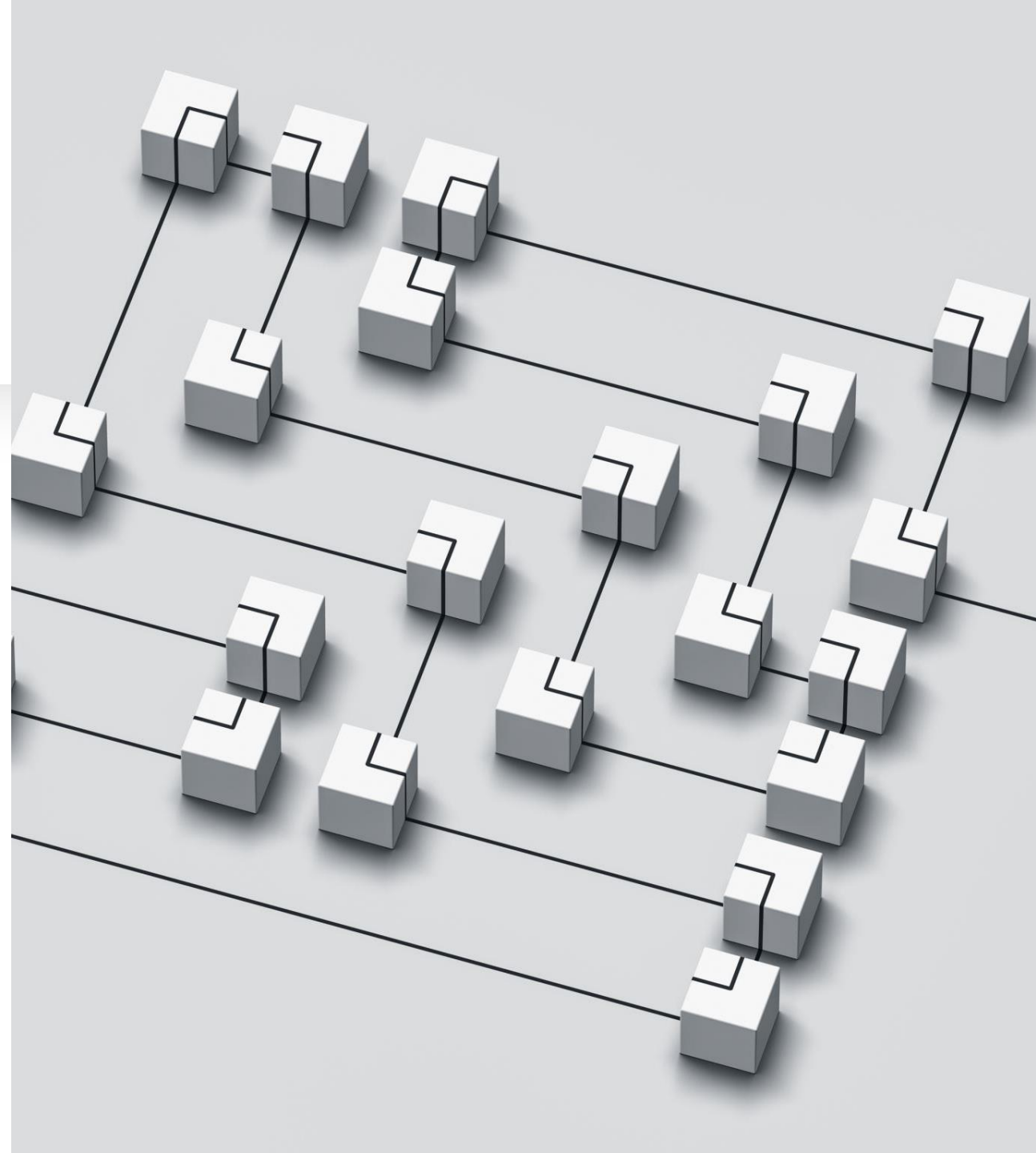
Metric / Aspect	Blueprint Target	Achieved Build Result	Status
Architecture	MobileNetV2 Transfer Learning	MobileNetV2 Fine-Tuned	Met
Test Accuracy	$\geq 85.0\%$	92.5%–95.0%	Exceeded
Data Balance	10 Balanced English Classes	500 images/class (\$5,000\$ clean)	Met
Inference Speed	<100 ms / image	15–25 ms / image	Exceeded
Failure Analysis	Analyze top visual errors	Exported Confusion Matrix & Error Plots	Met

What Changed From the Blueprint

- Engineering Decisions & Scope Adjustments
- Dataset Sampling Capping: * *Change*: Sampled 500 images per class (5,000 total) rather than training on the full un-balanced ~26,100 raw images.
- *Reason*: Prevented class-imbalance bias, balanced gradient updates across all 10 classes, and eliminated memory timeouts during Google Colab fine-tuning.
- Automated Label Pipeline: * Added a python string translation parser directly into data loading to handle raw Italian directory names seamlessly.

Challenges and Fixes

- Key Technical Obstacles
- Non-English Dataset Directory Names: * *Issue*: Kaggle folder titles were in Italian (gallina, scoiattolo, pecora).
 - *Fix*: Programmed an automated mapping dictionary (ITALIAN_TO_ENGLISH) prior to Keras image loading.
- Model Overfitting During Fine-Tuning: * *Issue*: Training accuracy rapidly reached 98%+ while validation loss began to creep upward.
 - *Fix*: Integrated spatial augmentation layers, added 20% Dropout, and reduced the fine-tuning learning rate to 10^{-5} .



Lessons and Next Steps

- Insights & Future Roadmap
- Key Lessons Learned: * Dataset curation, clean translation, and equal class distribution yield bigger accuracy gains than adding model complexity.
 - Fine-tuning only the top layers with a small learning rate prevents catastrophic forgetting of ImageNet features.
- Next Steps: * Export model weights to TensorFlow Lite (TFLite) for edge deployment on mobile/IoT devices.
 - Build a web-based UI (e.g., Streamlit or Gradio) for drag-and-drop user testing.